The Inner Workings of the CQRS Pattern

A Journey of a Command

A command signifies an intention to change the state of our system. Let's follow the journey of a command to understand how CQRS processes it.

Imagine a user wants to update their address in an e-commerce application. The command would be something like 'Update User Address'. This command would trigger a chain of events within the system.

Validation

Before anything else, the command is validated. This validation can involve checks for data format, permissions, and other business rules. In our example, is the new address in a valid format? Does the user have the necessary permissions to change their address? If the command passes these validations, it moves on to the next stage; otherwise, it's rejected.

Handling the Command

Once validated, the command is handled by the command handler. This component is the workhorse of the command side of CQRS. It's responsible for updating the state of the system based on the command. For instance, it updates the user's address in the system.

Generating Events

Every command leads to a change in state, represented by an event. In our case, the event could be 'User Address Updated'. This event is stored in the event store, serving as a historical record of all changes in the system.

The Query's Quest

A query, in essence, is a request for data. Let's say our user now wants to view their updated address. They issue a query: 'Get User Address'. How does CQRS handle it?

Unlike commands, queries don't require validation or event generation. They're simply processed by the query handler, which fetches the required data. In this case, it fetches the user's address from the system and displays it to the user.

Synchronizing Commands and Queries

We've explored the separate journeys of commands and queries. But how do these two sides stay synchronized? This is where Event Sourcing comes into play.

Remember the 'User Address Updated' event generated when the user updated their address? This event is replayed on the query side to update its state. In other words, the user's updated address is reflected in the query model. Thus, Event Sourcing ensures that despite working separately, the command and query sides are always in sync.